

채팅 에이전트 구축 방법론

5단계 프레임워크 — 구현 가이드

텐소트웍스 · 2026 · v3.0

핵심 공식

에이전트 = MCP(서비스 완전 커버) × 역할(명확한 목표) × 컨텍스트(실시간 상황) × 피드백(자기 진화)

실습 시나리오

이 문서 전체에서 "프로젝트 관리 서비스의 PM 에이전트"를 구축하는 과정을 예시로 사용합니다.
사용자가 Slack/채팅으로 "이번 주 마감 태스크 정리해줘", "클라이언트 미팅 잡아줘"와 같은 요청을 하면 에이전트가 프로젝트 관리 서비스에서 직접 실행하는 시나리오입니다.

제0 원칙

에이전트의 본질은 Write다

Read만 하는 AI는 검색엔진이다. 챗봇이다. 백과사전이다. "일정이 뭐 있어?"에 답하고, "이 문서 요약해줘"를 처리하는 것 — 이걸 에이전트가 아니다.

Write가 되는 순간 에이전트가 된다. 일정을 등록하고, 문서를 작성하고, 주문을 넣고, 상태를 변경하는 것. 사용자가 "해줘"라고 했을 때 **실제로 해지는 것**. 그게 에이전트다.

따라서 아래 5단계 전체가 "Write를 얼마나 완전하게 구현하느냐"를 향해 수렴한다.

▶ READ-ONLY 챗봇 VS WRITE 에이전트



"내일 클라이언트 미팅 잡아줘, 오후 2시"



Read-only: "네, 내일 오후 2시에 미팅을 잡으시려면 캘린더 앱에서 새 이벤트를 생성하시고..."



Write 에이전트: "내일 14:00 클라이언트 미팅 등록했어요. 회의실 B 예약하고, 참석자에게 초대 메일도 보냈습니다." calendar.create • room.reserve • email.send

1 Write 오퍼레이션 전수 조사

타겟 서비스에서 사용자가 수행하는 모든 Write 행위를 먼저 매핑한다. 생성, 수정, 삭제, 상태 변경, 전송 — 서비스의 상태를 바꾸는 모든 행위.

Read는 Write를 보조하기 위해 따라온다. "일정을 등록하려면 기존 일정을 조회해야 한다"는 식으로, Write에 필요한 Read를 역산해서 붙이는 구조.

핵심 질문: "이 서비스에서 사용자가 '직접 손으로 해야 하는 일'이 뭔가?" — 그게 전부 Write이고, 그걸 에이전트가 대신 하는 게 목표다.

구현 예시 — 프로젝트 관리 서비스 오퍼레이션 매핑

도메인	Write 오퍼레이션	보조 Read	타입
태스크	태스크 생성	프로젝트 목록 조회	WRITE
	상태 변경 (todo → in_progress → done)	태스크 상세 조회	WRITE
	담당자 배정	팀원 목록 조회	WRITE
	태스크 삭제	—	WRITE
일정	미팅 생성	빈 시간 조회	WRITE
	일정 수정/취소	일정 상세 조회	WRITE
	참석자 추가/제거	팀원 목록 조회	WRITE
문서	회의록 작성	미팅 정보 조회	WRITE
	주간보고서 생성	완료 태스크/커밋 조회	WRITE
알림	팀원에게 메시지 전송	—	WRITE

Check: 서비스 UI를 열어서 클릭할 수 있는 버튼을 전부 세라. 버튼 하나 = Write 하나. 빠짐없이 매핑되었는지 대조한다.

2 MCP 도구 래핑

1단계에서 나온 Write 오퍼레이션 전부를 MCP(Model Context Protocol) 도구로 구현한다.

- 원칙: 도구 하나 = Write 하나
- 우선순위: Write 도구 먼저, Read 도구는 그 다음
- 처음부터 전부 만들 필요 없이 핵심 Write 도구 10~20개로 시작
- "이것도 해줘"가 나올 때마다 추가하는 점진적 확장

도구 설계의 기준: "에이전트가 이 도구로 서비스 페이지를 거치지 않고 직접 결과를 만들어낼 수 있는가?"

```
// Write 도구: 태스크 생성
server.tool("create_task", "새로운 태스크를 생성합니다", {
  project_id: z.string().describe("프로젝트 ID"),
  title:      z.string().describe("태스크 제목"),
  assignee:   z.string().optional().describe("담당자 이름"),
  priority:   z.enum(["low", "medium", "high", "critical"]).optional(),
  due_date:   z.string().optional().describe("마감일 (YYYY-MM-DD)"),
}, async ({ project_id, title, assignee, priority, due_date }) => {
  const task = await db.tasks.insert({
    project_id, title, assignee, priority, due_date,
    status: "todo",
    created_at: new Date()
  });
  return { content: [{ type: "text", text: `태스크 생성 완료: ${task.id}` }] };
});

// Write 도구: 태스크 상태 변경
server.tool("change_task_status", "태스크 상태를 변경합니다", {
  task_id: z.string(),
  status:  z.enum(["todo", "in_progress", "done", "blocked"]),
}, async ({ task_id, status }) => {
  await db.tasks.update(task_id, { status });
  return { content: [{ type: "text", text: `상태 변경: ${status}` }] };
});

// Read 도구: 태스크 목록 (Write를 보조)
server.tool("list_tasks", "태스크 목록을 조회합니다", {
  project_id: z.string().optional(),
  status:     z.string().optional(),
  assignee:   z.string().optional(),
}, async (filters) => {
  const tasks = await db.tasks.find(filters);
  return { content: [{ type: "text", text: JSON.stringify(tasks) }] };
});
```

패턴: 도구 이름은 동사_명사 형태 (`create_task` , `update_schedule`). Write 도구는 변경 결과를, Read 도구는 데이터를 반환한다. 도구 설명(description)은 LLM이 도구를 선택하는 기준이 되므로 명확하게 작성한다.

3 역할(Goal) 정의

가장 중요하지만 가장 간과되는 단계. 같은 Write 능력이라도 **역할 정의에 따라 에이전트의 행동이 완전히 달라진다.**

정체성

에이전트가 누구인가

사용자 프로필

누구를 위해 Write하는가

자율 판단 범위

어떤 Write를 승인 없이 해도 되는가

커뮤니케이션 스타일

Write 전후로 어떻게 소통하는가

역할이 없으면 Write 능력이 있어도 판단 없는 도구 모음에 불과하다.

▶ 구현 예시 — 시스템 프롬프트 (역할 정의)

정체성

당신은 "파이(Pi)"입니다. [A사]의 프로젝트 매니저 에이전트로, 팀 리더에게 프로젝트 현황을 보고하고 실행을 지원합니다.

사용자 프로필

팀 리더(박민수)는 3개 프로젝트를 동시 관리하며, 디테일보다 "지금 뭐가 막혀있는지"를 빠르게 파악하길 원합니다.

자율 판단 범위

- 승인 없이 가능: 태스크 상태 변경, 일정 조회, 주간보고 생성
- 확인 후 실행: 태스크 삭제, 마감일 변경, 팀원에게 메시지 발송
- 선제적 행동: 마감 3일 전 알림, 2주 이상 정체 태스크 경고

커뮤니케이션 스타일

- 간결한 존댓말, 핵심 먼저
- 숫자 기반 보고 (완료율, 잔여 태스크 수, 지연일)
- 문제 보고 시 원인 + 제안을 함께 제시

위험 판단 기준

다음 상황을 위험으로 판단하고 선제적으로 보고:

- 마감일 3일 이내 미완료 태스크
- 2주 이상 상태 변화 없는 태스크
- 한 사람에게 태스크 5개 이상 집중

핵심: 같은 도구 세트를 "비서"로 정의하면 시킨 Write만 한다. "PM"으로 정의하면 병목을 감지해서 선제적으로 보고한다. "코치"로 정의하면 업무 패턴을 분석해서 개선을 제안한다. **역할이 행동을 결정한다.**

4 컨텍스트 주입 설계

Write의 품질은 컨텍스트가 결정한다. 같은 "일정 등록"이라도 현재 일정, 마감 현황, 대화 맥락을 알면 **충돌을 피하고 적절한 판단을 내린다.**

설계 기준: "**이 역할이 올바른 Write를 하려면 뭘 알고 있어야 하는가?**"

컨텍스트 없는 Write는 위험하다 — 잘못된 데이터를 쓰고, 충돌하는 일정을 만든다. Read만 하는 시스템은 틀려도 불편한 정도지만, Write하는 에이전트가 틀리면 **실제 피해가 생긴다.**

오늘 일정 (3건)

+

이번 주 마감 태스크 (7건)

+

프로젝트 진행률

+

이전 대화 요약

→

LLM 프롬프트

```
// 매 대화마다 실시간 상황을 주입
const context = {
  today:      "2026-04-08 (수)",
  schedules: await mcp.call("list_schedules", { week: "current" }),
  urgent:     await mcp.call("list_tasks",
    { due_before: "3_days", status: "!done" }),
  projects:  await mcp.call("list_projects", { status: "active" }),
  history:    getRecentConversation(10), // 최근 10턴
};

// 시스템 프롬프트에 삽입
const systemPrompt = `
${roleDefinition}

# 현재 상황
오늘: ${context.today}
이번 주 일정: ${JSON.stringify(context.schedules)}
긴급 태스크: ${JSON.stringify(context.urgent)}
프로젝트 현황: ${JSON.stringify(context.projects)}

# 이전 대화
${context.history}
`;
```

원칙: 컨텍스트는 역할에 맞춰서 설계한다. PM 에이전트에게 재무 데이터는 불필요하다. 주입 데이터가 너무 많으면 LLM이 혼란스러워지고 비용이 올라간다. "이 역할의 판단에 필수적인 최소 데이터"를 기준으로 잡는다.

일반 챗봇과 에이전트를 가르는 **결정적 차이**. 에이전트는 시간이 갈수록 더 정확하게 Write한다.



프롬프트 자기 수정

사용자 피드백을 즉시 영구 반영. "일정 등록할 때 알림도 넣어줘" → 이후 모든 일정에 자동 적용.



지식 축적

도메인 지식을 구조화 저장. "A클라이언트는 수요일 미팅 선호" 같은 학습이 쌓임.



도구 확장

새로운 Write 니즈 발견 시 MCP 도구 추가. 에이전트 실행 범위가 계속 넓어짐.

구현 예시 — 프롬프트 자기 수정



"일정 잡을 때 회의실도 같이 잡아줘. 매번 내가 따로 하는 거 귀찮아"



알겠어요. 앞으로 일정 등록 시 회의실 자동 예약 추가할게요. ✅ 행동 규칙 업데이트

```
// 피드백 감지 시 행동 규칙을 영구 저장
await db.agent_rules.upsert({
  rule_id: "schedule_creation",
  version: 2,
  content: "일정 생성 시 회의실 자동 예약 포함. 회의실 미지정 시 참석 인원 기준 적절한 회의실 자동 배정.",
  reason: "사용자가 매번 수동 예약하는 불편함 지적 (2026-04-08)",
});

// 다음 대화부터 이 규칙이 컨텍스트에 주입됨
const rules = await db.agent_rules.getAll();
systemPrompt += `\n# 행동 규칙\n${rules.map(r => r.content).join('\n')}`;
```

구현 예시 — 지식 축적 (온톨로지)

```
// 대화에서 사업 지식을 자동 추출
// 사용자: "A사 담당자 김부장은 수요일 오후만 미팅 가능해"

await knowledge.upsertEntity({
  name: "김부장",
  type: "person",
  properties: {
    organization: "A사",
    title: "부장",
    meeting_preference: "수요일 오후"
  }
});

// 이후 "A사 미팅 잡아줘" 요청 시
// → 자동으로 수요일 오후로 제안
```

```
// 사용자가 반복적으로 "주간보고 써줘"를 요청
// → 새 Write 도구 추가

server.tool("generate_weekly_report",
  "이번 주 완료/진행/지연 태스크를 종합해 주간보고서를 생성합니다",
  {
    project_id: z.string(),
    week_start: z.string().optional(),
  },
  async ({ project_id, week_start }) => {
    const completed = await getTasksByStatus(project_id, "done", week_start);
    const inProgress = await getTasksByStatus(project_id, "in_progress");
    const blocked = await getTasksByStatus(project_id, "blocked");
    const commits = await getCommits(project_id, week_start);

    const report = await llm.generate({
      prompt: `주간보고서 작성: 완료 ${completed.length}건, 진행중 ${inProgress.length}건, 차단 ${blocked.length}건, 커밋 ${commits.length}건`,
      data: { completed, inProgress, blocked, commits }
    });

    await db.documents.create({
      type: "weekly_report",
      content: report,
      project_id
    });

    return { content: [{ type: "text", text: report }] };
  }
);
```

진화 루프: 사용자가 에이전트를 쓸수록 (1) 행동 규칙이 정교해지고 (2) 도메인 지식이 쌓이고 (3) 할 수 있는 일이 늘어난다. 이 세 가지가 동시에 돌아갈 때 에이전트는 시간이 갈수록 사람에 가까워진다.

빠진 요소	결과
Write 부족	"못 합니다" 챗봇 — 대화는 되지만 실행이 안 됨
역할 부재	판단 없는 도구 모음 — 시키는 것만 하고 맥락을 모름
컨텍스트 부재	위험한 Write를 하는 사고 머신 — 틀린 데이터가 실제 피해로
피드백 부재	같은 실수 반복하는 정적 시스템 — 성장하지 않음

실전 적용 로드맵

Week 1~2: 1단계(Write 전수 조사) + 2단계(핵심 MCP 도구 10~20개 구현)

Week 3: 3단계(역할 정의) + 4단계(컨텍스트 주입 설계) — 고객과 함께 정의

Week 4~: 5단계(피드백 루프) — 운영하면서 자동 축적. 도구는 점진적으로 확장

1~2단계는 **개발팀**이, 3~4단계는 **고객과 함께** 정의하고, 5단계는 **운영하면서 자동으로 축적되는** 구조로 가는 것이 현실적이다.